COMP5434 Project Task 2: Review Rating Prediction with Spark Data Processing and an Ensemble of Text Encoders

Abstract

To predict ratings for approximately 7.68 million reviews, we progressively refined our approach around data quality, text representation, and computational efficiency. We first built a Spark pipeline for cleaning, sparse text features, and entity statistics. Reducing the nominal text dimensionality to 1/16 of its original value yielded predictive performance close to that of the higher-dimensional model, while four Spark threads sped up a fixed data-processing task by approximately 3.10×. We then traced high-error examples, identified rating inconsistencies, recovered ratings from the public source, and shifted our focus to fine-tuning text encoders that retain contextual information. A weighted ensemble of MiniLM, DistilBERT, and BGE with isotonic calibration achieved an RMSE of 0.49893 on the Kaggle public leaderboard. Keeping just MiniLM and BGE yielded a public RMSE of 0.49914, a difference of only 0.00021, and cut the same-architecture CPU inference benchmark time by approximately 61%. After finalizing the modeling approach, we used Spark to preprocess the full raw review dataset. We handled missing values, combined titles and review bodies, joined source ratings, and organized records into datasets ready for final model training and testing. Full preprocessing took 61.435 seconds, or 66.985 seconds including independent verification of all texts. Our approach uses experiments to identify the main bottlenecks and target improvements in supervision, text representation, and deployment design, balancing prediction quality against computational cost.

1. Problem Definition and Research Approach

Task 2 requires predicting ratings from 1 to 5 based on reviews and their associated attributes while balancing accuracy with large-scale data-processing efficiency. We formulate it as a regression task with continuous predictions in [1,5] and evaluate them using root mean squared error.

$$\mathrm{RMSE} = \sqrt{[(1/\mathrm{N}) \times \Sigma_{\mathrm{i}}(\hat{\mathrm{y}}_{\mathrm{i}} - \mathrm{y}_{\mathrm{i}})^2]}.$$

Because squared error amplifies large prediction errors, increasing model complexity alone is insufficient. We also need to assess whether the training ratings are reliable, whether key information in the reviews is retained, and whether additional computation yields tangible gains.

We first used Spark to establish a scalable data-processing and feature baseline and studied the data and computational costs experimentally. We then used error analysis to improve supervision and adopted pretrained text encoders to learn context. Finally, we used ensembling, calibration, and member pruning to improve prediction quality and deployment efficiency. The four-member ensemble is our final accuracy-focused solution, and the two-member version is its efficiency-focused refinement.

2. Data Analysis and Initial Exploration

2.1 Data Sources and Main Characteristics

We used kagglegit to download the original data from the course's Kaggle Task 2 competition, including train.csv, test.csv, and the sample submission. Each record contains a title, review body, user and product identifiers, timestamp, helpful-vote count, and purchase status. The training set also contains rating. We retained the original files so that later processing outputs could be traced to the experimental records.

Data characteristic	Analysis result
Training reviews	7,684,645
Test reviews	10,000
Duplicate training IDs	0
Missing bodies (train / test)	9,046 / 8
Invalid helpful-vote counts	1 record, originally −1
5-star reviews in original training set	4,253,020 (approx. 55.34%)
Mean characters (title / body)	**21.19 / 152.72**
Maximum characters (title / body)	519 / 26,255

These statistics directly informed our design. Five-star reviews make up a large share of the data, so relying on the overall mean can obscure low ratings. The model therefore needs to learn evaluative cues from the reviews. Titles usually summarize the reviewer's attitude, while the body describes the experience and specific reasons. We also applied consistent rules to the few missing or anomalous values to prevent field-level issues from affecting the models. These findings led us to prioritize text while investigating numeric and entity features in the initial experiments.

2.2 Data Processing and Feature Baseline with Spark

We used Spark DataFrames to clean and construct features for millions of reviews. Missing review bodies were replaced with empty strings and flagged. Negative helpful-vote counts were set to 0 and flagged as anomalous. Timestamps and purchase status were converted to consistent types. This produced usable inputs while preserving information about anomalous records.

We detected duplicates using the title, body, user, product, parent product, timestamp, helpful-vote count, and purchase status together, so identical short reviews would not be removed solely because they shared the same text. The early pipeline removed 28,518 records with identical inputs and conflicting ratings and 25,157 redundant duplicates with matching ratings, leaving 7,630,970 records. A deterministic stratified split with parameter 42 produced 6,104,774 training records and 1,526,196 validation records.

We designed three input groups to capture evaluative content, review context, and entity coverage.

Feature group	Design	Purpose
Basic numeric features (11)	Title and body lengths and word counts, helpful votes and their log, purchase status, year and month, missing-value and anomaly flags	Capture review length and context
Two sparse text channels	HashingTF, IDF, and L2 normalization applied separately to titles and bodies	Retain lexical cues with low vocabulary overhead
Entity statistics (9)	Review counts, log counts, and unseen flags for users, products, and parent products	Describe entity activity and training coverage

IDF and entity statistics were fitted on the training split alone. Validation and test data reused the same parameters. The entity features use counts, keeping mean ratings for each entity out of this feature group. We used these inputs in a hybrid regression model combining a sparse-text EmbeddingBag, a numeric branch, and a wide path.

2.3 Findings from Feature Compression

The original text dimensions were 65,536 for titles and 262,144 for review bodies. To reduce model parameter and storage costs, we tried 4,096 and 16,384 dimensions, respectively, reducing the total nominal dimension from 327,680 to 20,480, or 1/16 of the original size.

Early approach	Total text dimensions	Validation RMSE
Training mean baseline	—	1.514879
High-dimensional hybrid model	327,680	1.157312
Low-dimensional hybrid model	20,480	1.158683

The table uses competition ratings after the initial cleaning and 1,526,196 stratified validation records. The high- and low-dimensional models are reported from their respective runs.

This result shows the value of controlling feature size. The low-dimensional model retained nearly the same predictive ability in a much smaller representation space. Additional epochs also brought limited gains. Over three epochs, the low-dimensional model achieved validation RMSEs of 1.164079, 1.158683, and 1.159020, with the best result in the second epoch. We therefore shifted our focus from increasing feature size and training duration to data and semantic issues that more directly affected prediction error.

3. Key Findings and Development of the Final Solution

3.1 Identifying Rating Inconsistencies from High-Error Examples

While analyzing errors, we found reviews whose stated opinions clearly differed from their ratings. For example, a review titled “One Star” with the body “Yuck” had a rating of 5. We sought a traceable basis for the ratings before relying on sentiment rules to remove reviews. We checked the original CSV against the downloaded archive. The full SHA-256 hashes and file sizes matched, as did the ratings of the examples in question, confirming that the discrepancies predated our local cleaning.

We then identified the Grocery_and_Gourmet_Food category in McAuley Lab's Amazon Reviews 2023 dataset. We matched reviews using fingerprints of their bodies and timestamps to recover their public source ratings. This grounded our rating treatment in the original source and avoided changing training labels on linguistic intuition alone.

3.2 Recovering Source Ratings and Improving Supervision

We used 64-bit blake2b fingerprints of review bodies to identify candidates, then filtered them by timestamp. We skipped short reviews with too many candidates and stored each recovered rating and match status by its original competition ID. After verification, the full rating table contained 7,108,463 valid source ratings, covering approximately 92.50% of the training records. Agreement between source and competition ratings was approximately 80.0103%.

The ratings differed in approximately 19.99% of matched records. If a rating is replaced with probability p by a value drawn uniformly from 1 to 5, the following holds.

$$\mathrm{P}(\text{ratings differ}) = \mathrm{p} \times 4/5, \text{giving } \mathrm{p} \approx 0.1999 \div 0.8 \approx 0.2499.$$

This distribution is consistent with approximately 25% random rating replacement and gives us a statistical basis for prioritizing supervision. We trained the final model on valid source ratings and retained

competition ratings for traceability. When the supervision targets themselves show substantial discrepancies, establishing a consistent rating reference is a better use of resources than further increasing model size.

3.3 From Lexical Cues to Contextual Learning

Early sparse representations captured keywords efficiently but struggled to represent word order, negation, degree, and contrast. For example, “not good” and “good” share an evaluative word but convey different meanings. A review that says “It tastes great, but the packaging is damaged” also requires considering several aspects together. We therefore adopted pretrained text encoders so that the learned representations could use contextual relationships directly.

We retained both the title and the body. The title summarizes the evaluation, while the body describes the experience, and their combination gives the model a fuller basis for prediction. Unlike the normalization used for early bag-of-words features, the final input preserves original capitalization, punctuation, whitespace, and line breaks so that potentially useful cues reach the encoder. Each model's tokenizer handles tokenization and length limits. This input design reflects how contextual encoders use text.

We then compared prediction calibration, frozen representations, and encoder fine-tuning. Kaggle public leaderboard RMSE improved from 0.65882 with bag-of-words and isotonic calibration to 0.61998 with the frozen MiniLM combination, then to 0.51355 with the early fine-tuned MiniLM combination. These results supported using fine-tuned text encoders as the core of the final solution and exploring how multiple models could complement one another.

4. Final Solution Design and Implementation

4.1 Overall Architecture and the Role of Spark

The final solution combines data preparation, text representation learning, and prediction fusion. We use source ratings as a consistent supervision target, fine-tune encoders to learn review semantics, and combine and calibrate their predictions to produce final ratings. Spark processes inputs at scale and PyTorch trains the models. The two stages share review IDs and a common text specification.

Once the final input specification was set, we implemented it in Spark. The implementation preserves meaning, aligns reviews with ratings, and produces data that the final training code can read directly. Spark replaces missing titles and bodies with empty strings, concatenates them as title + ". " + comment, joins source ratings by ID, counts eligible records, and writes ordered output. We use consistent CSV parsing rules so that long reviews containing line breaks remain intact.

The implementation retains all 7,684,645 training texts and 10,000 test texts. Training eligibility is then determined by source-rating match status. We independently read back and checked every text, ID, and record position, as well as every entry in the exported rating array. The results matched the final model's input specification, establishing the link between Spark's parallel processing and the text modeling pipeline.

4.2 Sample Organization and Validation Design

We reserve records satisfying id % 40 == 0 for validation and use the remaining valid matches for training. This lets us compare all encoders on the same data. IDs are used only for joining, splitting, and ordering records and are excluded from the text model's inputs.

Final inputs and sample groups	Records
All training texts	7,684,645
Valid source ratings	7,108,463
Fixed validation split	192,117
Validation records with valid source ratings	177,607
Training records eligible for fine-tuning	6,930,856
Test records	10,000

Spark writes training and test text as ID-sorted Parquet files and exports the source-rating files. PyArrow packages the data in a streaming manner into the format read by the model. Across all records, there were 0 mismatches between original and archived ratings, invalid IDs, or missing joins. These checks safeguard model input quality and keep training and ensembling aligned to the same records.

4.3 Encoder Fine-Tuning and Regression

We chose MiniLM, DistilBERT, and BGE-small based on text representation ability and training and inference costs. MiniLM provides lightweight semantic encoding, while DistilBERT and BGE provide different pretrained representations. Different configurations of MiniLM further diversify predictions.

Each model connects the first-token representation h from the encoder's final layer to a linear regression head.

$$\hat{y} = w^{T}h + b.\ L = (1/N) \times \Sigma_i(\hat{y}_i - y_source_{,i})^2.$$

We update the regression head and encoder together to adapt the text representation directly to rating prediction. We use AdamW with a learning rate of 2×10^{-5} and weight decay of 0.01, warm up for the first 5% of steps before switching to a cosine schedule, cap the gradient norm at 1.0, and use CUDA mixed precision to support large batches.

Member	Encoder	Epochs	Max. tokens	batchsize	Single-model validation RMSE
s1	all-MiniLM-L6-v2	2	96	512	0.5428
s2	all-MiniLM-L6-v2 (different seed and configuration)	2	128	1,024	0.5506
d1	distilbert-base-uncased	2	128	768	0.5418
b1	BAAI/bge-small-en-v1.5	2	128	768	0.5328

Single-model results were computed on 177,607 matched validation records. BGE performed best and provided the strongest contribution to prediction. We assessed the value of retaining the other models based on the ensemble results.

4.4 Weighted Ensembling and Rating Calibration

Ensembling lets us take advantage of complementary prediction errors. We assign each model a nonnegative weight, with all weights summing to 1, instead of using a simple average. We first clip each model's predictions to [1,5], then compute the weighted prediction.

$$p = \Sigma_k \alpha_k \times \text{clip}(\hat{y}_k, 1, 5), \text{ where } \alpha_k \geq 0 \text{ and } \Sigma_k \alpha_k = 1.$$

Using a fixed random seed, we searched 400 sets of Dirichlet weights and selected a combination based on RMSE on the matched validation records. The saved weights are approximately 0.1773 for s1, 0.0051 for s2, 0.2558 for d1, and 0.5618 for b1. The submission script uses three decimal places.

The weighted ensemble stabilizes predictions. Isotonic regression then adjusts how predicted scores map to actual ratings. We chose a monotonic mapping to preserve rating order while correcting systematic bias across score ranges. Calibration was fitted on the even-positioned half of the matched validation sequence and evaluated on the odd-positioned half. We applied the fixed weights and mapping to test predictions.

The four-member weighted ensemble achieved a validation RMSE of approximately 0.5276, a calibration evaluation RMSE of 0.5245, and a Kaggle public leaderboard RMSE of 0.49893. We therefore selected it as the final accuracy-focused solution.

4.5 Deployment Optimization Based on Member Contributions

After selecting the accuracy-focused solution, we asked whether each member justified its deployment cost. The final weight of s2 was small, DistilBERT took longer for CPU forward passes, and MiniLM and BGE already complemented each other well. We therefore retained s1 and b1 for the two-member version.

We searched two-member weights on the calibration fitting half in steps of 0.01, obtaining 0.33 for MiniLM and 0.67 for BGE, then fitted the isotonic mapping. Member pruning reused existing trained models without additional fine-tuning. It changes the ensemble composition while preserving each retained member's architecture and parameter precision.

This design bases pruning on evidence of each member's contribution. We retain the complementary information most useful for rating prediction and use the prediction results to guide efficiency improvements.

5. Performance Evaluation and Discussion

5.1 Evolution of the Solution and Kaggle Evaluation

The table summarizes representative experiments from the source-rating modeling stage. Kaggle submission scores provide the main evidence of final predictive performance, and local validation results help assess individual design choices.

Approach	Local validation RMSE (source ratings)	Kaggle public RMSE
Bag-of-words prediction + affine calibration	0.7515	0.69326
Bag-of-words prediction + isotonic calibration	0.7043	0.65882
Frozen MiniLM representations + bag of words + isotonic calibration	0.6789	0.61998
Early fine-tuned MiniLM configuration + bag of words + isotonic calibration	0.5399	0.51355
Final weighted four-member ensemble + isotonic calibration	0.5245	0.49893
Optimized two-member ensemble (s1 + b1) + isotonic calibration	0.5255	0.49914

Local scores for the four- and two-member ensembles come from the calibration evaluation half and are development results on the validation set. This split and the early baseline split in Section 2.3 correspond to their respective experimental setups.

These results show how our optimization priorities evolved. Isotonic calibration improved the rating mapping, frozen encoders added semantic information, and end-to-end fine-tuning adapted the representations directly to rating prediction, yielding a larger gain. Compared with the frozen representation combination, the early fine-tuned combination reduced public leaderboard RMSE from 0.61998 to 0.51355, a relative reduction of approximately 17.2%. The final ensemble reached 0.49893, reflecting the combined effects of better supervision, task adaptation, and complementary errors.

The main finding is that identifying data problems and selecting a task-appropriate representation was more effective than simply increasing feature dimensionality or training duration. The early Spark experiments clarified the data-processing and computational costs, and later solutions focused computation on stages that improved predictions.

5.2 Member Comparison and Effects of Pruning

Member combination	Isotonic calibration RMSE	Kaggle public RMSE	Members
s1	0.5377	—	1
b1	0.5271	—	1
s1 + d1, equal weights	0.5302	—	2
s1 + b1, weights 0.33 / 0.67	0.5255	0.49914	2
s1 + s2 + d1 + b1	0.5245	0.49893	4

Single- and two-member comparisons used the same odd-positioned evaluation half, containing 88,799 matched records. Two-member weights and mappings were determined on the even-positioned half. The four-member ensemble uses the weight search and calibration setup in Section 4.4.

MiniLM with BGE outperformed both individual members in the table and the equally weighted MiniLM and DistilBERT combination. This suggests that complementarity matters more than the number of models. The two-member public leaderboard RMSE was only 0.00021 higher than that of the four-member ensemble. It substantially simplified deployment while retaining comparable prediction quality, supporting our contribution-based selection.

5.3 Computing Resources and Runtime Environment

We assigned resources according to the workload. Spark ran large-scale preprocessing on one local machine, Google Colab handled encoder fine-tuning, and CPU resources handled source-rating recovery and prediction fusion. Cloud experiments used 3 runtime instances or sessions in total, with 1 single-GPU instance per model training job. The table lists the GPUs, vCPUs, RAM, and VRAM for each instance.

Stage	CPU / GPU	Server and thread settings / compute instances and parallel resources	RAM / VRAM
Spark preprocessing and early feature experiments	Intel Core i7-12700H	1 local machine, 14 cores, 20 logical threads, 4 Spark threads in the final run	Approx. 15.7 GB RAM, 4 GB driver
Source-rating recovery, fusion, and CPU benchmark	Intel Core i7-1260P	1 machine, 16 threads	7 GB RAM
s1 training	NVIDIA L4	Single-GPU cloud instance, 12 vCPUs	52 GB RAM, 23,034 MiB VRAM
s2, d1, and b1 training	NVIDIA RTX PRO 6000 Blackwell	Single-GPU instance per job, 48 vCPUs	176 GB RAM, 97,887 MiB VRAM

The Spark environment used Windows, Python 3.11.8, Java 21.0.7, PySpark 4.0.1, NumPy 2.4.6, and PyArrow 26.0.1. Final preprocessing used local[4] and 32 shuffle partitions. Source-rating recovery and prediction fusion used Python 3.10.12, PyTorch 2.14.1+cpu, Transformers 5.20.2, NumPy 2.2.6, scikit-learn 1.7.2, and PyArrow 25.0.1. Cloud training records correspond to Python 3.13 and Transformers 5.x.

5.4 Training Efficiency and Cumulative Training Time

All four encoders were trained for two epochs. We use measured process times and logged throughput to report computation time per epoch and total training process time.

Model	batchsize	Steps per epoch	Training compute per epoch (s)	Measured two-epoch process (s)
s1	512	13,537	Approx. 3,813	7,992
s2	1,024	6,769	Approx. 1,132	2,264
d1	768	9,025	Approx. 1,872	3,869
b1	768	9,025	Approx. 1,674	3,484

Steps per epoch are the current eligible training size divided by the batch size, rounded up. Per-epoch times for s1, d1, and b1 are estimated from the step counts and logged throughput. For s2, we divide the two-

epoch process time by two. The measured process times include overhead such as validation during training and saving. The cumulative process time across all four models was 17,609 seconds, or approximately 4.89 hours.

BGE achieved the best single-model validation RMSE of 0.5328 with a two-epoch process time of 3,484 seconds, earning the highest ensemble weight. This suggests that fine-tuned lightweight encoders can deliver competitive rating predictions, and that adding a few complementary members better serves the efficiency goal than increasing model size indiscriminately.

5.5 Spark Preprocessing and Total Offline Time

Full Spark preprocessing covers approximately 7.68 million training reviews and all test reviews. It includes text construction, source-rating joins, quality checks, and model input file generation. It took 61.435 seconds, or 66.985 seconds with independent verification of all texts.

Preprocessing stage	Measured time (s)
Prepare source-rating table	0.933
Start Spark session	4.252
Read training data, build text, join ratings, and verify	30.193
Sort, write, and package training text	18.792
Write and package rating results	6.111
Export model rating array and verify consistency	0.207
Read test data, build text, and verify	0.395
Write and package test text	0.175
Full preprocessing, including array loading and call overhead	61.435
Total including independent verification of all texts	66.985

Preprocessing time includes Spark startup, computation, and file generation, starting with the prepared competition CSVs and the full source-rating table. We use these measurements to recalculate total offline time.

$$T_offline = T_preprocessing + T_training + T_ensemble.$$

Offline workload	Cumulative time (s)	Approx. hours
Reuse source-rating table: 61.435 + 17,609 + 3.30	17,673.735	4.91
Above, with independent verification of all texts	17,679.285	4.91
Recover source ratings: 155 + 61.435 + 17,609 + 3.30	17,828.735	4.95
Include source download: 245 + 155 + 61.435 + 17,609 + 3.30	18,073.735	5.02

The table adds the recorded times for each stage. Preprocessing uses the new Spark measurement, while training and ensemble calibration use the existing measurements from their corresponding experiments. Public source-rating recovery and download took 155 seconds and approximately 245 seconds, respectively. Reusing the rating table avoids another recovery run. The new Spark step replaces the original text construction step, so the offline total counts that work only once.

Once an effective modeling approach is established, Spark can process large-scale inputs efficiently using local resources. Encoder fine-tuning accounts for most of the offline computation, so further optimization should prioritize training configurations and ensemble composition.

5.6 Inference Cost for All Test Records

To estimate deployment cost, we ran a CPU benchmark on all 10,000 test texts, covering model loading, tokenization, forward passes, prediction fusion, calibration, and CSV output. Each four- and two-member configuration used the corresponding input lengths and batch sizes.

Four-member stage	**Time (s)**
s1 forward pass, max. length 96, batch 512	36.45
s2 forward pass, max. length 128, batch 1,024	42.85
d1 forward pass, max. length 128, batch 768	162.62
b1 forward pass, max. length 128, batch 768	138.07
Load tokenizers and models	17.43
Tokenize for all models	3.89
Fuse, calibrate, and write CSV	1.53
Sum of stages	402.84 (approx. 403)
Two-member stage	**Time (s)**
s1 loading, tokenization, and forward pass	49.1
b1 loading, tokenization, and forward pass	108.0
Read, fuse, calibrate, and write CSV	0.08
Total	**157.18 (approx. 157.2)**

The two benchmarks ran independently on an i7-1260P with 16 threads and 7 GB of RAM. They used pretrained base weights with the same architectures and input configurations as the final models to approximate deployment cost after fine-tuning. Reconstructing a submission from saved predictions was separately recorded at 1.53 seconds. That time is for submission reproduction and excludes encoder forward passes.

Comparing approximately 403 seconds with 157.2 seconds gives an approximately 2.56× speedup and a time reduction of about 61% for the two-member solution. With a public leaderboard RMSE difference of just 0.00021, member pruning preserves most predictive performance while reducing the cumulative cost of loading, tokenization, and forward passes.

5.7 Spark Parallelism and Early Feature Efficiency

In early feature exploration, we fixed the task to reading the training split, computing basic features, and aggregating them, then compared 1, 2, and 4 Spark threads. Each configuration had one warm-up and three timed runs. We use the median to represent typical task execution time.

Spark threads	Three runs (s)	Median (s)	Speedup vs. one thread
1	79.44 / 119.22 / 83.47	83.47	1.00×
2	44.05 / 43.46 / 42.59	43.46	1.92×
4	28.55 / 26.89 / 26.97	26.97	3.10×

Four-thread parallel efficiency was approximately 77.4%, showing that the task used local parallel resources effectively. The low-dimensional hybrid model took 214.34, 220.69, and 230.24 seconds across three epochs, totaling 665.33 seconds. Together with text representations at 1/16 of the nominal dimension, these results illustrate how the early solution controlled feature size and computation.

These experiments informed the final allocation of resources. Spark handles scale, task-specific fine-tuning improves semantic representations, and member selection controls deployment cost. Parallel measurements refer to the fixed feature task after warm-up. The final input processing time comes from the full run in Section 5.5.

6. Summary and Future Work

We began with Spark cleaning, sparse text, and entity statistics. The dimensionality experiments demonstrated the feasibility of feature compression, and the parallel experiments showed that millions of reviews could be processed locally. Error analysis then shifted our focus to rating provenance and contextual representation. We recovered valid source ratings and centered the final model on learning semantics from review titles and bodies.

Encoder fine-tuning, weighted ensembling, and isotonic calibration produced a final Kaggle public leaderboard RMSE of 0.49893. The two-member variant reached 0.49914 and reduced the same-architecture CPU benchmark time by approximately 61%. Spark preprocessing using the final input specification took 61.435 seconds, or 66.985 seconds including full-text verification, providing a defined input pipeline and measured evidence of its efficiency.

Our main lesson is to consider accuracy and efficiency together. Data quality determines how useful supervision is, text representations determine what information a model can interpret, and ensemble composition and parallel processing determine how computation is spent. Our experiments identified priorities in all three areas, allowing data analysis to guide model selection and prediction results to guide deployment optimization.

Future work will prioritize knowledge distillation from the four-member ensemble to one model, as well as quantization and batch processing optimizations for the existing encoders, building on the accuracy and cost benefits of pruning. The data preparation, training, and ensemble modules retain fixed configurations. The test output is checked for 10,000 records, ID order, and rating range to provide a consistent basis for further experiments.

7. References and Experimental Records

[1] COMP5434 Project, course project brief, 2026. This report covers Dataset 2 / Task 2.

[2] McAuley Lab, Amazon Reviews 2023, Grocery_and_Gourmet_Food public review dataset. https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023

[3] Team records for models, ensemble pruning, and Kaggle submissions, FINETUNE_RESULT.md, repository commit 15951cebf9f05f1aad0d8e31cfca7e6a1bf46d97. https://github.com/PKLgames/comp5434project/blob/15951cebf9f05f1aad0d8e31cfca7e6a1bf46d97/task2_model/FINETUNE_RESULT.md

[4] Team records for hardware, training times, and inference times, ENV_AND_TIMING.md, same model experiment version. https://github.com/PKLgames/comp5434project/blob/15951cebf9f05f1aad0d8e31cfca7e6a1bf46d97/task2_model/ENV_AND_TIMING.md

[5] Final input specification and source-rating table, DATA_SPEC.md and clean_y.npz, repository commit 261e9e648cc0b1080fae0fe192f9ab1fa7635946. https://github.com/PKLgames/comp5434project/blob/261e9e648cc0b1080fae0fe192f9ab1fa7635946/task2_model/audit/DATA_SPEC.md

[6] Team records for full Spark preprocessing, 2026-11-09. The entry point is spark_preprocessing/04_preprocess_full.py. Timing, statistics, and full consistency check results are in spark_preprocessing/output/spark_full_20261109_120930_513308/preprocessing_summary.json.

Appendix: Team Contributions

Name	Student ID	Actual contributions across both tasks	Team contribution share
LIANG Guirong	26102379g	Task 2: Handled data analysis, Spark preprocessing, and text and entity feature engineering. Ran feature compression and parallel efficiency experiments, built and verified the final model inputs, and compiled preprocessing timings, reproducibility code, the README, and the report.	25%
CHEN Wanchuan	26108204g	Task 2: Handled source-rating recovery, model design, and training. Ran experiments on encoder fine-tuning, weighted ensembling, isotonic calibration, and member pruning. Generated predictions, submitted them to Kaggle, evaluated performance, and documented model configurations, results, and runtimes.	25%
XU Kesen	—		
WANG Ruixian			
		Jointly discussed the approach, analyzed results, and integrated the deliverables.	100%